

EECS 482

Pointers and Makefile

Agenda

- Void Pointers
- Function Pointers
- Makefile

— — —

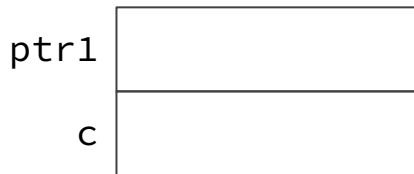
Void Pointers

Pointers

- A pointer is a variable that stores the address to another variable
- Examples:
 - `char *`
 - `int *`
 - `MyCustomClass *`
- With each the above examples, there is a **type** associated with the pointer (`char`, `int`, `MyCustomClass`)

Pointers are variables that hold memory addresses

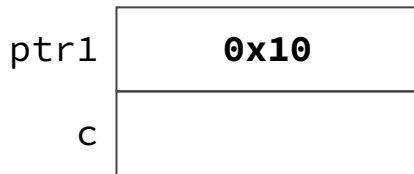
```
char *ptr1 = 0x10;  
char c = *ptr1;  
cout << c << endl; // a  
  
int *ptr2 = 0x14;  
int x = *ptr2;  
cout << x << endl; // 482
```



Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

Pointers are variables that hold memory addresses

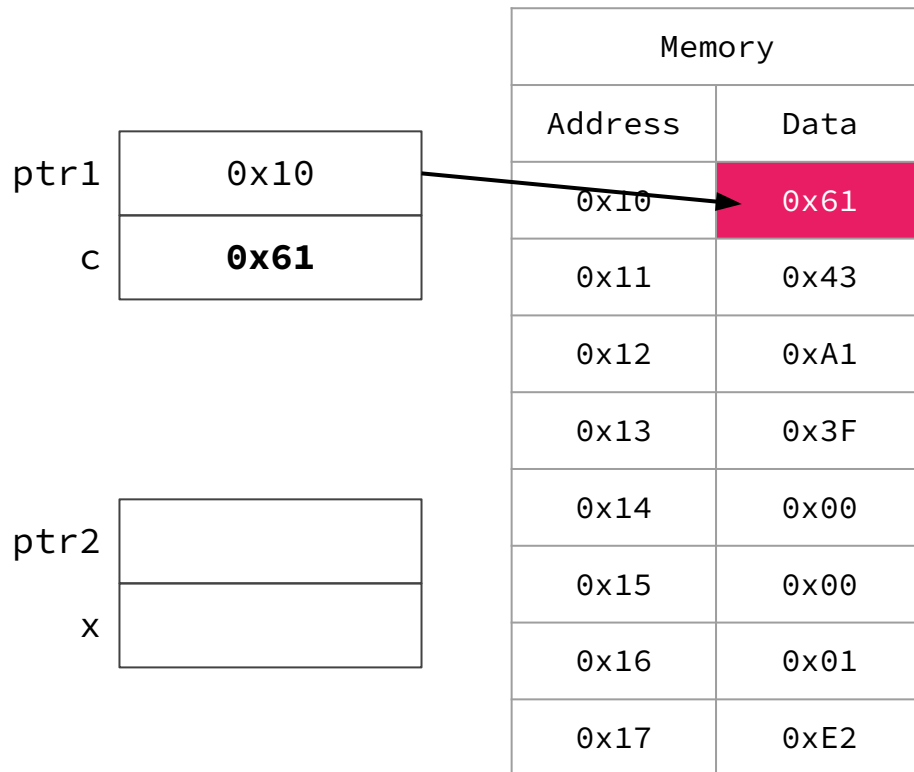
```
char *ptr1 = 0x10;  
char c = *ptr1;  
cout << c << endl; // a  
  
int *ptr2 = 0x14;  
int x = *ptr2;  
cout << x << endl; // 482
```



Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

Pointers are variables that hold memory addresses

```
char *ptr1 = 0x10;  
char c = *ptr1;  
cout << c << endl; // a  
  
int *ptr2 = 0x14;  
int x = *ptr2;  
cout << x << endl; // 482
```



Pointers are variables that hold memory addresses

```
char *ptr1 = 0x10;  
char c = *ptr1;  
cout << c << endl; // a  
  
int *ptr2 = 0x14;  
int x = *ptr2;  
cout << x << endl; // 482
```

ptr1	0x10
c	0x61

ptr2	
x	

Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

Pointers are variables that hold memory addresses

```
char *ptr1 = 0x10;  
char c = *ptr1;  
cout << c << endl; // a  
  
int *ptr2 = 0x14;  
int x = *ptr2;  
cout << x << endl; // 482
```

ptr1	0x10
c	0x61

ptr2	0x14
x	

Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

Pointers are variables that hold memory addresses

```
char *ptr1 = 0x10;  
char c = *ptr1;  
cout << c << endl; // a  
  
int *ptr2 = 0x14;  
int x = *ptr2;  
cout << x << endl; // 482
```

ptr1	0x10
c	0x61

ptr2	0x14
x	0x000001E2

Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

Pointers are variables that hold memory addresses

```
char *ptr1 = 0x10;  
char c = *ptr1;  
cout << c << endl; // a  
  
int *ptr2 = 0x14;  
int x = *ptr2;  
cout << x << endl; // 482
```

ptr1	0x10
c	0x61

ptr2	0x14
x	0x000001E2

Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

A pointer's type allows us to do pointer arithmetic

```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)
```

```
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```

ptr1



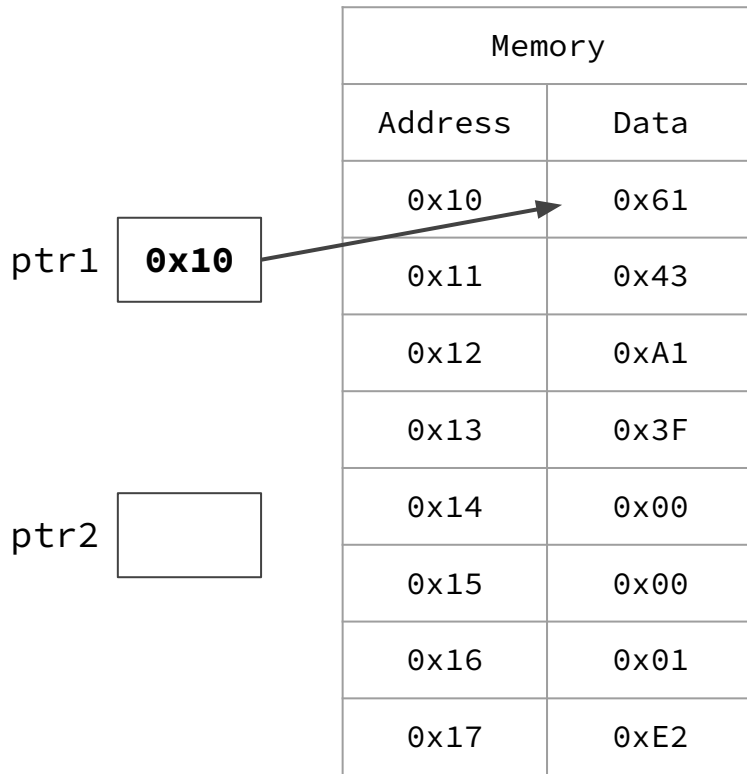
ptr2



Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

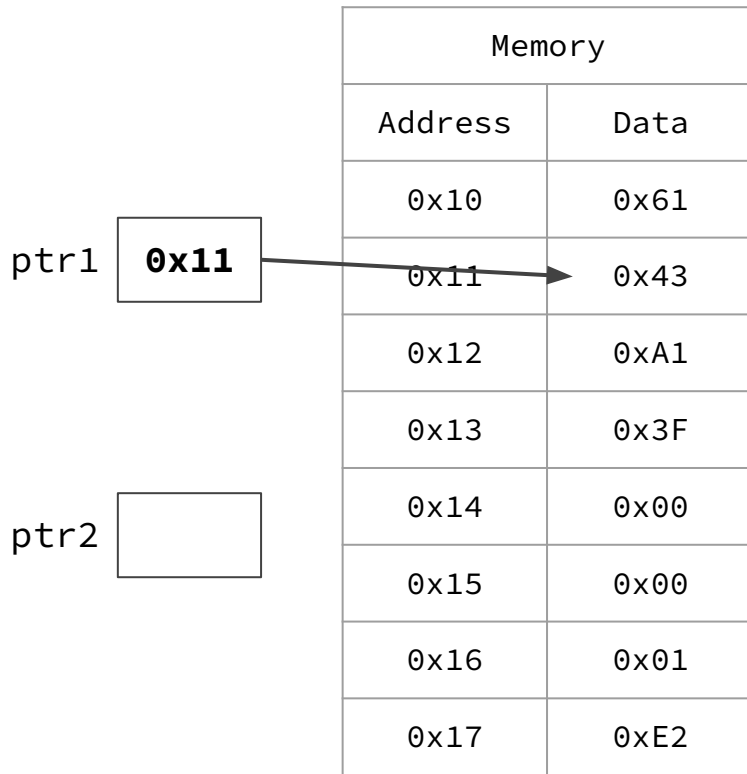
A pointer's type allows us to do pointer arithmetic

```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)  
  
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```



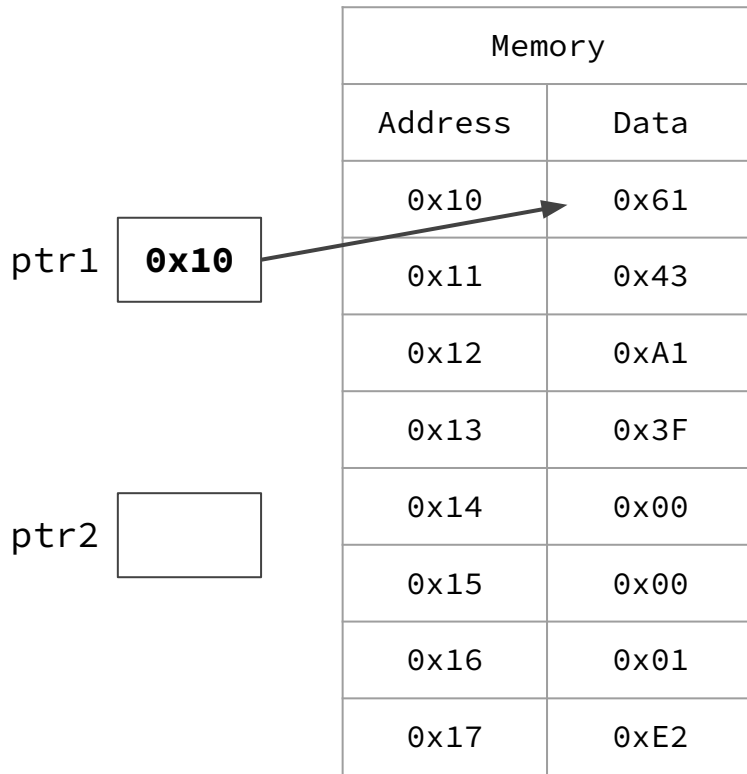
A pointer's type allows us to do pointer arithmetic

```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)  
  
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```



A pointer's type allows us to do pointer arithmetic

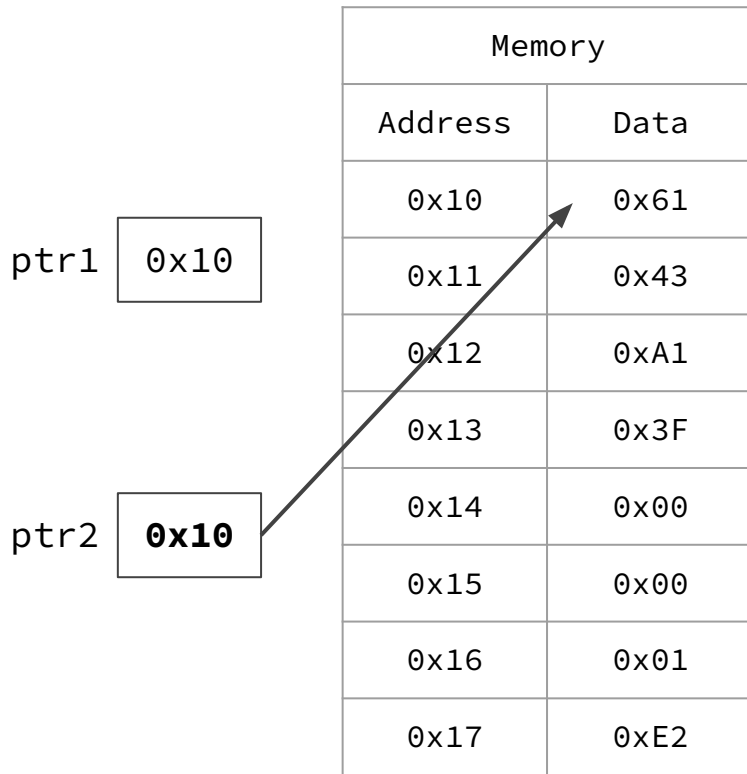
```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)  
  
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```



A pointer's type allows us to do pointer arithmetic

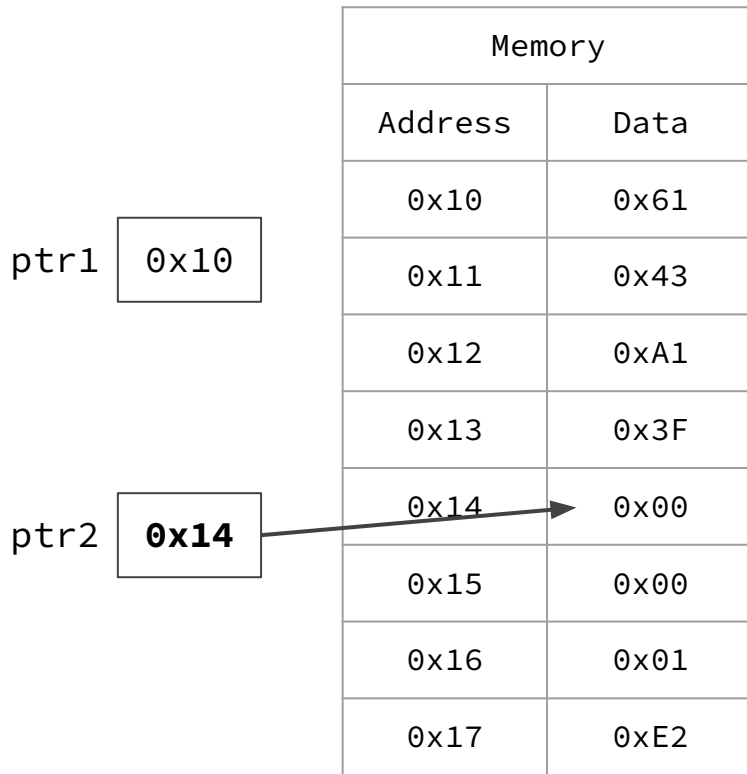
```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)
```

```
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```



A pointer's type allows us to do pointer arithmetic

```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)  
  
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```



A pointer's type allows us to do pointer arithmetic

```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)  
  
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```

ptr1

0x10

ptr2

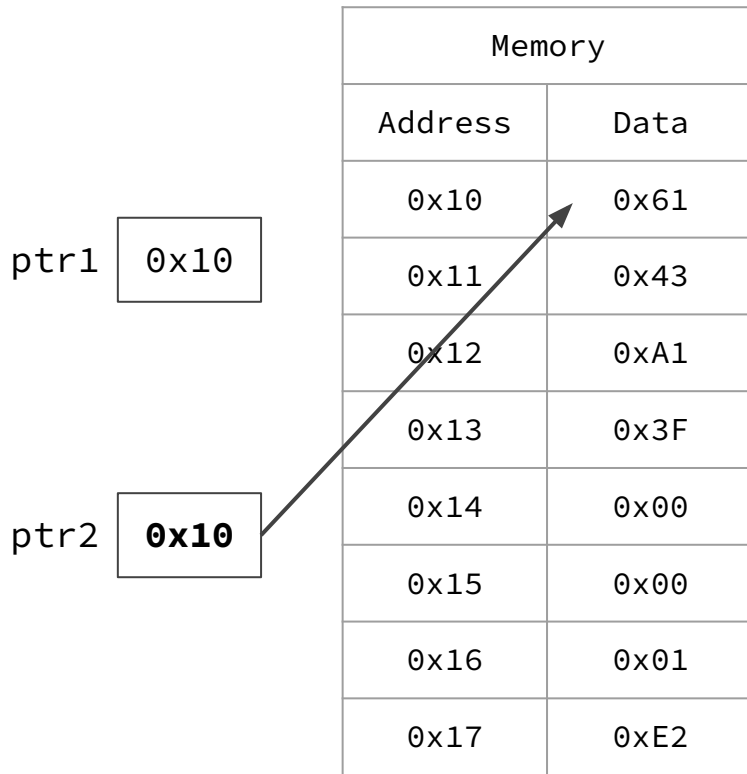
0x14

The actual value
of the pointer
increased by 4,
since an int is 4
bytes!

Memory	
Address	Data
0x10	0x61
0x11	0x43
0x12	0xA1
0x13	0x3F
0x14	0x00
0x15	0x00
0x16	0x01
0x17	0xE2

A pointer's type allows us to do pointer arithmetic

```
char *ptr1 = 0x10;  
++ptr1; // advance by one char (1 byte)  
--ptr1; // go back by one char (1-byte)  
  
int *ptr2 = 0x10;  
++ptr2; // advance by one int (4 bytes)  
--ptr2; // go back by one int (4 bytes)
```



Types of Pointers

- A pointer's type information also allows the compiler to do static type checking for us

```
void foo(char *c) {
```

```
    ...
```

```
}
```

```
int main() {
```

```
    int i = 482;
```

```
    foo(&i);
```

```
}
```

```
// note: candidate function not viable: no known conversion from 'int *' to 'char *'  
for 1st argument
```

Function Pointers

Function Pointers

- We've seen pointers being used as variables which contain the address of another variable
- Variables have addresses, but so do functions!
- So how do we store the address of a function? We can do so using **function pointers**
- With a function pointers, we can call the function being pointed to
- *Function pointers allow us to write generic code for functions, just as void pointers allow us to write generic code for normal variables*

Function Pointer Syntax

- For the compiler to generate the code for a call to the function using the function pointer, it needs to know what the return type and parameter types are
- Syntax:
 - `return_type (*function_ptr_variable_name)(type1, type2 etc);`
- Examples:
 - `int (*foo)(int, double);`
 - `void (*bar)(void *);`
 - `void* (*baz)();`

Function Pointer Syntax

- For the compiler to generate the code for a call to the function using the function pointer, it needs to know what the return type and parameter types are
- Syntax:
 - `return_type (*function_ptr_variable_name)(type1, type2 etc);`
- Examples:
 - `int (*foo)(int, double);`
 - `void (*bar)(void *);`
 - `void* (*baz)();`

Pointer to a function that takes an int and a double and returns an int.

Function Pointer Syntax

- For the compiler to generate the code for a call to the function using the function pointer, it needs to know what the return type and parameter types are
- Syntax:
 - `return_type (*function_ptr_variable_name)(type1, type2 etc);`
- Examples:
 - `int (*foo)(int, double);`
 - `void (*bar)(void *);`
 - `void* (*baz)();`

Pointer to a function that takes a void pointer and returns nothing.

Function Pointer Syntax

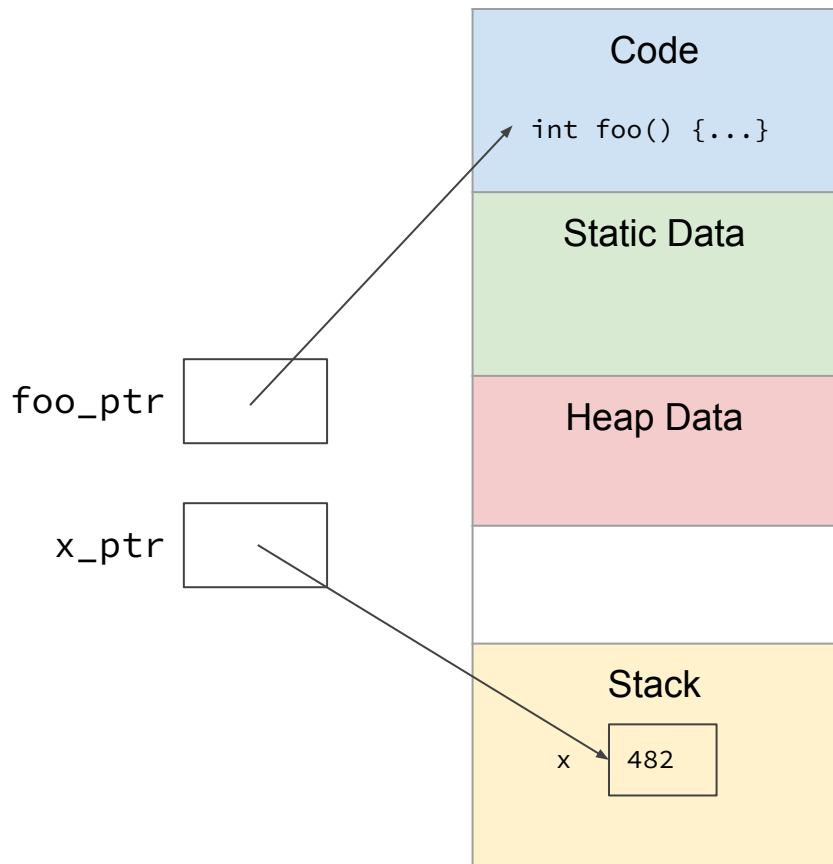
- For the compiler to generate the code for a call to the function using the function pointer, it needs to know what the return type and parameter types are
- Syntax:
 - `return_type (*function_ptr_variable_name)(type1, type2 etc);`
- Examples:
 - `int (*foo)(int, double);`
 - `void (*bar)(void *);`
 - `void* (*baz)();`

Pointer to a function that takes nothing and returns a void pointer.

Example

Example

```
int foo() {  
    return 482;  
}  
  
int main() {  
    int (*foo_ptr)();  
    foo_ptr = foo;  
    int x = foo_ptr();  
    int *x_ptr = &x;  
}
```



Some more syntax

- Cast syntax: `reinterpret_cast<return_type (*)>(type1, type2, etc)>`
 - Omit the name, leave the `*`
 - See previous slide for an example
- using syntax: `using func_type_name = return_type (*)>(type1, type2)`
 - Use the “using” syntax rather than “typedef”
 - The first two lines of `main()` from the previous slide can be simplified:

```
void* baz() {...}

...

using baz_fcn_t = void* (*);
baz_fcn_t baz_ptr = reinterpret_cast<baz_fcn_t>(baz); // explicit cast
baz_fcn_t baz_ptr2 = baz; // implicit cast
```

Why are function pointers needed for EECS 482?

- Project 1:
 - `using thread_startfunc_t = void (*) (uintptr_t);`
 - `thread(thread_startfunc_t func, uintptr_t arg);`
 - `static void boot(thread_startfunc_t func, uintptr_t arg, unsigned int deterministic);`
- The function signatures of `cpu::boot` and the `thread` constructor (which are functions you will need to call) require you to **pass functions** in the form of function pointers

References/Resources

- http://websites.umich.edu/~eecs381/lecture/C_Coverage.pdf

Makefile

Makefiles

— — —

What are they?

Files containing directives to automate compilation and other command-line tasks, **ordered by dependencies**

Learn this because:

Typing out (or finding in history)
`g++ scheduler.cpp -o scheduler.exe...`
is a waste of time and error-prone

They're really powerful (can implement a personal autograder with them)

They're ubiquitous

Makefiles: targets/recipes

— — —

- Makefiles contain a list of targets and their recipes
- A recipe is a list of shell commands that are run when **make <target>** is typed in the terminal

```
target:
    command1
    command2
```



TABS not spaces!

Makefiles: target example

— — —

- Use a Makefile to easily switch between git users on the same computer
- Running **make manusk** will configure git for Manos, and **make bnoble** will do the same for Brian
- Remember, Makefile recipes needn't be compilation commands, though they often are

manusk:

```
git config user.name "Manos Kapritsos"  
git config user.email manusk@umich.edu
```

bnoble:

```
git config user.name "Brian Noble"  
git config user.email bnoble@umich.edu
```

Makefiles: dependencies

— — —

- A dependency can be a filename or another target
- A target runs if: (a file with the target's name doesn't exist) **OR** (a dependency file has been modified more recently than the target's file)
- A target runs its dependencies' recipes before its own

```
target: dep1 dep2
    command1
    command2
```



TABs not spaces!

Makefiles: dependency example

— — —

```
# Write sample.txt with contents of depends.txt
sample.txt: depends.txt
    cat depends.txt > sample.txt
```

\$ touch depends.txt	# create empty file
\$ make sample.txt	
cat depends.txt > sample.txt	
\$ make sample.txt	
make: 'sample.txt' is up to date.	# no changes since last make
\$ echo "Hello, world!" >> depends.txt	# write to depends.txt
\$ make sample.txt	
cat depends.txt > sample.txt	

Makefiles: variables

— — —

- Variables are a list of strings for use in recipes
- Text functions are run when **make** is run, and can be used to interface with filenames in the file system

```
TESTS    = $(wildcard test*.cpp)
SOURCES  = $(filter-out $(TESTS), $(wildcard *.cpp))
HEADERS  = $(wildcard *.h)
OBJECTS  = $(SOURCES:.cpp=.o)
```

Makefiles: Some more shortcuts



564



`$$` is the name of the target being generated, and `$<` the first prerequisite (usually a source file). You can find a list of all these special variables in the [GNU Make manual](#).

For example, consider the following declaration:

```
all: library.cpp main.cpp
```



In this case:

- `$$` evaluates to `all`
- `$<` evaluates to `library.cpp`
- `$$` evaluates to `library.cpp main.cpp`

Share Edit Follow Flag

edited Aug 21 '20 at 21:14



Steve Lorimer

22.1k ● 13 ● 92 ● 176

answered Jul 10 '10 at 17:59



bdonlan

201k ● 27 ● 241 ● 314